

IEEE 1687 / 1838 DFT Insertion Flow

insert_dft_with_dwr.tcl -- Single-Pass RTL Guide

Scope: a single Tessent Shell (RTL-stage) script that inserts Main TAP, HostIjtag, HostStap, and IEEE 1838 DWR (dedicated wrapper cells) in ONE process_dft_specification call, run from golden functional RTL.

Document version: v2.0 -- supersedes the earlier two-phase (RTL then gate-level) version.

This revision replaces the previous approach, which inserted DWR at gate level, after synthesis, via analyze_wrapper_cells + insert_test_logic in a separate Phase 2. That approach is NOT used here. This version follows the RTL-stage wrapper insertion approach instead, per the Tessent Shell User's Manual section "Wrapper Analysis and Dedicated Wrapper Cell Insertion at RTL".

1. Why this must be a single pass, from golden RTL

- Tessent RTL mode runs a quick internal synthesis to perform wrapper analysis, and process_dft_specification inserts the dedicated wrapper cells directly into the RTL -- this happens at the same stage as Main TAP / HostIjtag / HostStap insertion, not after.
- set_dedicated_wrapper_cell_options on -ports forces specific ports to receive a dedicated wrapper cell during that same insertion pass.
- If a third-party scan insertion flow is used later (for scan stitching / ATPG), Tessent records the wrapper cell information in dft_info_dictionary so it can be referenced downstream.
- You must start from **golden functional RTL** and run wrapper analysis + process_dft_specification together, in one spec. Taking RTL that already has Main TAP/HostIjtag/HostStap inserted and opening a second, separate spec just to bolt on wrapper cells does not work: Main TAP, STAP, IJTAG and wrapper control all need to come out of the **same** process_dft_specification call. Otherwise the IR ends up fixed before DWR control is added, and that combination cannot be verified cleanly.

1.1 Correct end-to-end order

- ① Golden functional RTL (no DFT logic yet)
- ② Tessent dft -rtl: wrapper analysis + forced dedicated wrapper cells, then HostIjtag + HostStap, in ONE process_dft_specification call (this script)
- ③ Tessent-inserted RTL (Main TAP + HostIjtag + HostStap + DWR wrapper cells, all present together)
- ④ Synopsys DC synthesis, using your existing standard-cell .db
- ⑤ Gate-level netlist
- ⑥ Functional + wrapper-control simulation
- ⑦ Later, separately: scan stitching / ATPG library integration

2. Section-by-section walkthrough

Section 0 -- User configuration

- TOP_DESIGN / RTL_SOURCES -- your die's top module and golden RTL file list. RTL_SOURCES must be the pre-DFT RTL, not anything with TAP/IJTAG/STAP already inserted.
- MAIN_TAP_PATH_TAIL / DESIGN_ID / TSDB_DIR -- same role as in your existing HostIjtag/HostStap flow.
- D2D_PORTS_LIST -- **the most important variable**: the real die-to-die functional ports. Must be replaced with your actual port names; do not include PTAP/STAP/TCK/TMS/reset pins.

Section 1 -- Load golden RTL

set_context dft -rtl, read the golden RTL files, set the current design, then
set_dft_specification_requirements -logic_test on to enable the logic-test infrastructure this flow needs.

Section 2 -- Wrapper analysis (before process_dft_specification)

Identifies the D2D ports, excludes everything else via set_wrapper_analysis_options -exclude_ports, forces dedicated wrapper cells with set_dedicated_wrapper_cell_options on, then runs check_design_rules and analyze_wrapper_cells. A report is written immediately (report_wrapper_cells_pre_spec.rpt) so you can confirm both D2D ports show dedicated wrapper status **before** continuing to the spec below.

Dedicated (rather than shared) wrapper cells are used here because they are the most reliable way to prove the flow end-to-end, especially when D2D outputs are purely combinational (e.g. driven directly by an ALU) with no nearby reusable flop.

Section 3 -- Main TAP + HostIjtag + HostStap, same spec

This is your already-validated create_dft_specification / read_config_data / process_dft_specification sequence, reproduced unchanged -- except it now runs immediately after the wrapper analysis above, in the same Tessent Shell session, so that TAP, STAP, IJTAG and the DWR wrapper cells all come out of this single process_dft_specification call.

Section 4 -- Extract, save, and dump wrapper info

extract_icl and write_design save the combined result. dft_info_dictionary's dedicated_wrapper_cells entry is dumped to dwr_dedicated_wrapper_cells.rpt -- instance names here should match report_wrapper_cells_pre_spec.rpt from Section 2.

2.1 Sections 5-6 -- synthesis notes and success criteria

The rest of the script is comments only (no Tessent commands run here), covering what happens after this script finishes:

- Synthesis (Synopsys DC) runs outside Tessent Shell, on the RTL this script wrote out. DC only synthesizes the RTL; it does not need to understand IEEE 1838 semantics, same as your existing TAP/IJTAG synthesis.

- During synthesis: do not tie test-mode/shift/capture/update signals to constants, protect the Tessent DFT hierarchy, do not let constant propagation remove wrapper control paths, wrapper instance count must not decrease, no unresolved modules near D2D ports, and keep set_dont_touch scoped to the DFT hierarchy rather than the whole top or D2D port path.
- Recommended first target: basic_die_top with a 1-bit input + 1-bit output D2D pair, checked against the 7-item success checklist at the bottom of the script (wrapper ports listed, dedicated status, instances present after process_dft_specification, matching dft_info_dictionary entries, clean DC synthesis, matching mission-mode behavior, and controllable capture/shift/update).

3. Values you must replace before running

Variable	Placeholder value	What to do
TOP_DESIGN	current_die	Your top module name
RTL_SOURCES	./rtl/current_die.v	Golden RTL file list -- must have NO DFT logic inserted yet
DESIGN_ID	rtl2	Any id you like for this write_design pass
D2D_PORTS_LIST	up_d2d_in / up_d2d_out / down_d2d_in / down_d2d_out	Your real D2D functional port names; exclude any PTAP/STAP/TCK/TMS/reset pins

Before running, also confirm the exact syntax of the following commands against your installed Tessent version: set_dft_specification_requirements, set_wrapper_analysis_options, set_dedicated_wrapper_cell_options, analyze_wrapper_cells, create_dft_specification, read_config_data, process_dft_specification.

4. Full script (insert_dft_with_dwr.tcl)

Reproduced below for reference; identical to the attached .tcl file.

```
#####
# insert_dft_with_dwr.tcl
#
# Purpose:
#   Single-pass Tessent Shell (RTL stage) script that inserts, in ONE
#   process_dft_specification call:
#     - Main TAP
#     - HostIjtag
#     - HostStap
#     - IEEE 1838 DWR (dedicated wrapper cells) on the die-to-die (D2D)
#       functional ports
#
#   This follows the "wrapper analysis + dedicated wrapper cell insertion
#   AT RTL" approach described in the Tessent Shell User's Manual section
#   "Wrapper Analysis and Dedicated Wrapper Cell Insertion at RTL":
#     - RTL mode runs a quick internal synthesis to perform wrapper
#       analysis.
#     - process_dft_specification inserts the dedicated wrapper cells
#       directly into the RTL.
#     - set_dedicated_wrapper_cell_options on -ports forces specific
#       ports to get a dedicated wrapper cell.
#     - If a third-party scan insertion flow is used afterwards, Tessent
#       records the wrapper cell info in dft_info_dictionary.
#
#   MUST run from golden functional RTL. Do NOT take RTL that already has
#   Main TAP/HostIjtag/HostStap inserted and open a second spec just to
#   add wrapper cells -- Main TAP, STAP, IJTAG and wrapper control must
#   all come out of the SAME process_dft_specification call, or the IR
#   ends up fixed before DWR control is added, which cannot be verified
#   cleanly.
#
# Correct end-to-end order:
#   Golden functional RTL
#     -> Tessent dft -rtl : HostIjtag + HostStap + forced dedicated
#         wrappers, ONE process_dft_specification call
#     -> Tessent-inserted RTL
#     -> Synopsys DC synthesis with your existing standard-cell .db
#     -> Gate-level netlist
#     -> Functional + wrapper-control simulation
#     -> (later, separately) scan stitching / ATPG library integration
#
# Before running, verify the exact syntax of the following against your
# installed Tessent version:
#   help set_dft_specification_requirements
#   help set_wrapper_analysis_options
#   help set_dedicated_wrapper_cell_options
#   help analyze_wrapper_cells
#   help create_dft_specification
#   help read_config_data
#   help process_dft_specification
#####

#-----
# 0. USER CONFIGURATION -- the only section you should need to edit
#-----

set TOP_DESIGN          "current_die"                                ;# TODO

set RTL_SOURCES          [list "./rtl/current_die.v"]                ;# TODO: golden RTL only,
```

```

;# no DFT logic inserted yet

set MAIN_TAP_PATH_TAIL "IjtagNetwork/HostScanInterface(tap)/Tap(main)" \
;# usually unchanged

set DESIGN_ID "rtl2" ;# TODO: id this pass is
;# written out under

set TSDB_DIR "./tsdb_rtl_dwr"

# True die-to-die functional ports that need a DWR wrapper cell.
# List ONLY these -- do not include PTAP/STAP/TCK/TMS/reset or any other
# test pin; HostIjtag/HostStap already cover those.
set D2D_PORTS_LIST { \
    up_d2d_in \
    up_d2d_out \
    down_d2d_in \
    down_d2d_out \
} ;# TODO: your real D2D ports

#-----
# 1. Load golden RTL
#-----

set_context dft -rtl

set_tsdb_output_directory $TSDB_DIR

foreach f $RTL_SOURCES {
    read_verilog $f
}

set_current_design $TOP_DESIGN

set_dft_specification_requirements -logic_test on

#-----
# 2. Wrapper analysis setup -- MUST happen before process_dft_specification
#    so the dedicated wrapper cells get built into the same RTL insertion
#    pass as Main TAP / HostIjtag / HostStap.
#-----

set d2d_ports [get_ports $D2D_PORTS_LIST]

if {[sizeof_collection $d2d_ports] == 0} {
    puts "ERROR: no ports matched D2D_PORTS_LIST -- check section 0"
    return -code error "no matching D2D ports"
}

puts "INFO: the following ports will get a dedicated DWR wrapper cell:"
report_collection $d2d_ports

set non_d2d_ports \
    [remove_from_collection [get_ports *] $d2d_ports]

# Only analyze wrapper cells on the true D2D ports; everything else
# (PTAP/STAP/TCK/TMS/reset/etc.) is excluded.
set_wrapper_analysis_options \
    -exclude_ports $non_d2d_ports

# Forced dedicated wrapper cells: larger area, but the most reliable way
# to prove the flow end-to-end, especially if the D2D outputs are purely
# combinational (e.g. driven directly by an ALU) with no nearby reusable
# flop. Once the design has suitable pipeline/I/O flops you can revisit

```

```

# this and try shared wrapper cells instead.
set_dedicated_wrapper_cell_options on \
    -ports $d2d_ports

check_design_rules

set_system_mode analysis
analyze_wrapper_cells
report_wrapper_cells -verbose > report_wrapper_cells_pre_spec.rpt

puts "INFO: wrote report_wrapper_cells_pre_spec.rpt -- confirm both D2D"
puts "    ports show as dedicated wrapper, not excluded or optimized,"
puts "    before continuing."

# IMPORTANT: do NOT run `analyze_wrapper_cells -reset` between here and
# process_dft_specification below -- it would discard the wrapper
# analysis you just did.

#-----
# 3. Main TAP + HostIjtag + HostStap -- same spec, same
#    process_dft_specification call as the wrapper analysis above
#-----

set spec [create_dft_specification]

set main_tap_path "$spec/$MAIN_TAP_PATH_TAIL"

read_config_data \
    -in $main_tap_path \
    -from_string {
        HostIjtag(1) {
        }

        HostStap(s1) {
        }
    }

report_config_data $spec

process_dft_specification -validate_only
process_dft_specification

puts "INFO: Main TAP + HostIjtag + HostStap + dedicated DWR wrapper cells"
puts "    have all been inserted into the RTL by this single spec."

#-----
# 4. Extract ICL, save design, dump wrapper cell info
#-----

extract_icl
write_design -tsdb -design_id $DESIGN_ID -verbose

set dft_info [get_dft_info_dictionary]
if {[dict exists $dft_info dedicated_wrapper_cells]} {
    set fh [open "dwr_dedicated_wrapper_cells.rpt" w]
    puts $fh [format_dictionary [dict get $dft_info dedicated_wrapper_cells]]
    close $fh
    puts "INFO: wrote dwr_dedicated_wrapper_cells.rpt -- instance names"
    puts "    here should match report_wrapper_cells_pre_spec.rpt."
}

puts "INFO: RTL-stage insertion complete."
puts "NEXT STEP: synthesize this output with Synopsys DC using your"
puts "    existing standard-cell .db. See the checklist below"

```

```
puts "          for what to watch out for during synthesis, and the"
puts "          success criteria to confirm afterwards."
```

```
#####
# 5. Synthesis (Synopsys DC) -- runs OUTSIDE Tessent Shell, on the RTL
#   this script just wrote out. DC only needs to synthesize the RTL;
#   it does not need to understand IEEE 1838 semantics -- same as your
#   already-successful TAP/IJTAG RTL synthesis.
#
#   During synthesis, watch out for:
#       - Do not tie test-mode, shift, capture, or update control signals
#         to constants.
#       - Protect the Tessent-generated DFT hierarchy.
#       - Do not let constant propagation remove the wrapper control
#         paths.
#       - Wrapper cell instance count must not decrease after synthesis.
#       - No unresolved modules should appear near the D2D ports.
#       - Existing set_dont_touch can stay scoped to the Tessent DFT
#         hierarchy, but do not lock down the entire top or the whole
#         D2D port path.
#####
```

```
#####
# 6. First-pass success criteria
#   Recommended first target: basic_die_top with a 1-bit input + 1-bit
#   output D2D pair. Confirm, in order:
#
#   [ ] report_wrapper_cells lists both D2D ports
#   [ ] their status is "dedicated wrapper", not excluded or optimized
#   [ ] process_dft_specification produced wrapper instances in the RTL
#   [ ] dft_info_dictionary lists the same instances
#   [ ] DC synthesizes successfully with the existing .db, no
#       unresolved references
#   [ ] mission-mode functionality matches the original RTL
#   [ ] the wrapper can be exercised for capture, shift, and update
#       control behavior
#
#   Once items 5-7 above are confirmed, this can be reported as:
#       "Tessent has successfully generated dedicated DWR logic at RTL,
#       and the inserted architecture has been mapped to the existing
#       standard-cell library using Synopsys DC. Final scan stitching
#       and ATPG integration remain as the next step."
#####
```